

SoK: Hardware 기반 ML 모델 탈취 공격의 체계적 분류

SoK: A Systematic Classification of Hardware-Based ML Model Extraction Attacks

이다연 (Lee Dayeon)

고려대학교 사이버국방학과 (Department of Cyber Defense, Korea University)

Contents

1	서론 및 연구 동기	2
1.1	ML 모델의 지적재산으로서의 가치	2
1.2	하드웨어의 불가결한 역할	2
1.3	하드웨어 기반 탈취 공격의 실질성과 영속성	2
2	관련 연구 및 전체 조망	2
2.1	기존 체계화 연구와의 차별점	2
2.2	수집 논문 전체 목록	2
3	Taxonomy 제안	3
3.1	분류 축 설계	3
3.2	논문별 Dimension 매핑	3
3.3	분석 초점: Cache 및 Memory/Ciphertext 계열	4
4	실험 재현	4
4.1	재현 범위와 한계	4
4.2	Cache Telepathy	5
4.2.1	개요	5
4.2.2	재현 환경	5
4.2.3	진행 경과 및 공백 보완	5
4.2.4	결과 시각화	5
4.3	GANRED	5
4.3.1	개요	5
4.3.2	재현 환경	5
4.3.3	진행 경과 및 공백 보완	5
4.3.4	결과 시각화	5
4.4	BitShield	6
4.4.1	개요	6
4.4.2	재현 환경	6
4.4.3	진행 경과 및 공백 보완	6
4.4.4	결과 시각화	6
4.5	재현 과정에서의 발견	6
5	논의 및 SoK 기여	7
5.1	제안 Taxonomy의 의의	7
5.2	식별된 연구 공백	7
5.3	재현 실험이 드러낸 분류론적 함의	7
5.4	한계 및 향후 과제	7
6	결론	7

1 서론 및 연구 동기

1.1 ML 모델의 지적재산으로서의 가치

머신러닝 모델은 막대한 비용과 시간을 들여 만들어진 결과물이다. 대형 모델 하나를 학습시키는 데 수백만 달러의 연산 비용이 소요되며 [1], 여기에는 독점 데이터셋 구축과 수개월에 걸친 엔지니어링 노력이 포함된다. 이 때문에 ML 모델은 기업의 핵심 지적재산으로 취급된다.

이러한 가치는 자연스럽게 탈취 동기를 만든다. 모델 탈취(model stealing)란 피해자 모델의 내부 구조나 파라미터를 권한 없이 복원하는 행위로, 관련 위협은 이미 오래전부터 인식되어 왔다 [2]. 그러나 기존 체계화 연구들 [3]은 하드웨어 사이드채널 공격을 단일 범주로 처리하는 데 그친다. 공격마다 다른 누출 원천, 위협 모델, 탈취 가능 정보의 차이를 구분하지 않으면 정밀한 방어 설계가 어렵다. 세분화된 분류 체계가 필요한 이유다.

1.2 하드웨어의 불가결한 역할

보안 위협의 무게중심은 시대마다 바뀐다. 메모리 안전성 취약점은 논리 버그로, 네트워크 공격은 공급망 침해로 이동해왔다. 방어 기술이 발전하면서 특정 공격 표면은 좁아지기도 한다. 그러나 어떤 소프트웨어도 하드웨어 없이는 실행되지 않는다는 사실은 바뀌지 않는다. 연산은 반드시 물리적 실리콘 위에서 이루어지고, 데이터는 물리적 메모리 버스를 경유하며, 그 과정에서 타이밍 변동과 전력 소비 같은 부수 정보가 발생한다. 이 물리적 사실은 소프트웨어 패치로 제거할 수 없다.

오늘날 이 관찰은 더욱 중요해졌다. LLM 추론 수요가 폭발적으로 증가하면서 GPU, TPU, NPU 같은 전용 가속기가 AI 인프라의 핵심을 차지하게 되었다. 가속기가 다양해질수록 하드웨어 레이어의 누출 특성도 다양해지고, 그만큼 공격 표면은 넓어진다. 컴퓨팅 패러다임이 전환될수록 하드웨어 채널은 오히려 더 많아진다.

1.3 하드웨어 기반 탈취 공격의 실질성과 영속성

하드웨어 사이드채널을 이용한 ML 모델 탈취는 이론적 가능성이 아니다. Cache Telepathy [4]는 물리적 접근 없이 클라우드 동거 테넌트(co-located tenant) 권한만으로 동작하며, 상용 추론 서비스에 직접 적용 가능한 시나리오다. 소프트웨어 방어로 채널을 차단하기 어렵다는 점도 확인된다. HyperTheft [22]와 CipherSteal [23]은 신뢰실행환경(TEE)으로 모델을 암호화해도 암호문 사이드채널 분석으로 가중치와 입력 데이터를 탈취할 수 있음을 보였다. 모델이 관찰 가능한 하드웨어 위에서 실행되는 한, 채널은 존재한다.

본 보고서는 이러한 배경에서 출발하여 하드웨어 기반 ML 모델 탈취 공격의 체계적 분류를 시도하고, 그 과정에서 드러나는 연구 공백을 식별한다.

2 관련 연구 및 전체 조망

2.1 기존 체계화 연구와의 차별점

하드웨어 사이드채널을 이용한 ML 모델 탈취 연구는 2018년 이후 빠르게 축적되었으나, 이를 포괄적으로 정리한 체계화 연구는 드물다. 가장 근접한 선행 연구인 SoK: All You Need to Know About On-Device ML Model Extraction [3]은 탈취 결과와 접근 방식을 중심으로 taxonomy를 구성하되, 하드웨어 사이드채널을 단일 범주로 처리하는 데 그친다. 본 보고서는 하드웨어 채널 계보에 특화된 세분화된 분류를 제안하며, 기존 연구가 식별하지 못한 공백과 연구 흐름의 구조를 드러낸다.

2.2 수집 논문 전체 목록

표 1은 본 보고서에서 검토한 논문의 전체 목록이다. 누출 원천을 기준으로 분류하였으며, 방어 및 인접 연구는 별도 표기하였다. 색상은 채널 계열을 나타낸다.

Table 1: 하드웨어 기반 ML 모델 탈취 논문 전체 목록

논문	발표	채널	비고
DeepRecon [5]	2018	Cache (Flush+Reload)	최초 Cache SCA → DNN
Cache Telepathy [4]	USENIX SEC'20	Cache (Prime+Probe)	★ 핵심 논문
GANRED [6]	CCSW'20	Cache (Prime+Probe+GAN)	정확한 아키텍처 복원
DeepCache [7]	CCS'24	Cache (DL compiler)	DNN executable 대상
InputSnatch [8]	arXiv'24	Cache (KV-cache timing)	LLM 입력 추론
CSI NN [9]	USENIX SEC'19	EM + DPA	MCU, 가중치+아키텍처
DeepEM [10]	HOST'20	EM	FPGA
Maia et al. [11]	USENIX SEC'22	EM	GPU
Barracuda [12]	arXiv'23	EM	NVIDIA GPU 가중치
Wei et al. [13]	arXiv'18	Power	CNN 가속기
Open DNN Box [14]	TCAS-II'20	Power	아키텍처 · sparsity
Zhang et al. [15]	FPGA'21	Power (원격)	FPGA
DeepTheft [16]	S&P'24	Power (RAPL)	CPU, 원격
Hua et al. [17]	DAC'18	Memory access pattern	FPGA, 최초 HW SCA
DeepSniffer [18]	ASPLOS'20	DRAM bus snooping	GPU
Hermes Attack [19]	USENIX SEC'21	PCIe traffic	GPU, 아키텍처+가중치
DeepSteal [20]	S&P'22	Rowhammer (DRAM)	가중치 bit-flip 탈취
HuffDuff [21]	ASPLOS'23	Sparse accel timing	pruned DNN
HyperTheft [22]	CCS'24	Ciphertext SCA (TEE)	가중치 탈취
CipherSteal [23]	S&P'25	Ciphertext SCA (TEE)	입력 데이터 탈취
Compiled Models [24]	NDSS'25	Rowhammer	integrity attack
BitShield [25]	NDSS'25	-	방어 논문

3 Taxonomy 제안

3.1 분류 축 설계

본 보고서는 하드웨어 기반 ML 모델 탈취 공격을 다섯 개의 독립적 차원으로 분류한다. 각 차원은 공격의 서로 다른 속성을 포착하며, 조합을 통해 개별 공격의 위협 모델을 정밀하게 기술할 수 있다.

- **Dimension 1 - 누출 원천 (Leakage Source):** 공격이 관찰하는 물리적 채널이다. Cache / EM / Power / Bus · PCIe / Memory · Ciphertext의 다섯 범주로 구분된다. 이 축은 공격의 물리적 기반을 결정하며, 방어 기법의 적용 가능성과 직결된다.
- **Dimension 2 - 탈취 대상 (Extraction Target):** 공격이 복원하려는 정보다. 아키텍처(레이어 구조, 하이퍼 파라미터), 가중치(수치 값), 입력 데이터(사용자 쿼리), 무결성 파괴로 구분된다.

- **Dimension 3 - 위협 모델 (Threat Model):** 물리적 접근 필요 여부, 피해자 모델에 대한 사전 지식 수준 (white-box / grey-box / black-box), 능동적 쿼리 필요 여부, 공유 메모리 요구 여부를 포함한다.
- **Dimension 4 - 대상 하드웨어 (Target Hardware):** CPU / GPU / FPGA / MCU / TEE / NPU · 가속기 / DRAM으로 구분된다.
- **Dimension 5 - 복원 정밀도 (Recovery Precision):** 후보군 축소(search space reduction), 정확한 아키텍처 복원(exact recovery), 가중치까지 포함한 완전 복원(full model recovery)의 세 단계로 구분된다.

3.2 논문별 Dimension 매핑

표 2는 수집된 논문을 다섯 차원에 따라 매핑한 결과다. 행 색상은 표 1의 채널 계열과 동일하다.

Table 2: 논문별 Taxonomy 매핑

논문	Dim 1	Dim 2	Dim 3 (접근)	Dim 3 (쿼리)	Dim 4	Dim 5
Hua et al.'18	Bus/Mem	Architecture	물리	passive	FPGA	후보 축소
DeepRecon'18	Cache	Architecture	물리	passive	CPU	후보 축소
CSI NN'19	EM	Arch+Weights	물리	active	MCU	완전 복원
Cache Telepathy'20	Cache	Architecture	원격	passive	CPU	후보 축소
DeepSniffer'20	Bus	Architecture	물리	passive	GPU	후보 축소
GANRED'20	Cache	Architecture	원격	passive	CPU	정확 복원
Hermes'21	PCIe	Arch+Weights	물리	passive	GPU	완전 복원
DeepSteal'22	Rowhammer	Weights	원격	passive	DRAM	완전 복원
Maia'22	EM	Architecture	물리	passive	GPU	후보 축소
HuffDuff'23	Mem timing	Architecture	물리	passive	Sparse Accel	정확 복원
DeepTheft'24	Power	Architecture	원격	passive	CPU	정확 복원
DeepCache'24	Cache	Architecture	원격	passive	CPU	정확 복원
HyperTheft'24	Ciphertext	Weights	원격	passive	TEE	완전 복원
InputSnatch'24	Cache	Input	원격	passive	CPU	입력 추론
CipherSteal'25	Ciphertext	Input	원격	passive	TEE	입력 추론
BitShield'25	-	-	-	-	-	방어 논문
Compiled Models'25	Rowhammer	Integrity	원격	passive	DRAM	(파괴)

3.3 분석 초점: Cache 및 Memory/Ciphertext 계열

Dimension 1 전체 스펙트럼 중 본 보고서는 Cache SCA와 Memory · Ciphertext SCA 두 계열을 중심으로 분석한다. 이 선택은 세 가지 근거를 갖는다.

첫째, 두 채널 모두 물리적 접근 없이 원격 환경에서 동작한다. EM · Power · Bus 계열이 물리적 근접성이나 장비를 요구하는 것

과 달리, 이 두 채널은 클라우드 동거 테넌트 수준의 권한만으로 실행된다.

둘째, 현재 방어 기술의 최전선과 맞닿아 있다. TEE 기반 모델 보호가 보편화되는 시점에서, 암호화된 메모리 접근 패턴 자체를 채널로 삼는 Ciphertext SCA는 가장 최근의 공격 벡터다.

셋째, 두 계열은 탈취 대상(Dim 2)과 플랫폼(Dim 4+5) 조합에서 뚜렷한 구조적 패턴과 공백을 보인다. 표 3는 이를 정리한 것이다.

Table 3: Dim 1 필터(Cache · Memory/Ciphertext) 적용 후 Dim 2 × Dim 4+5 매핑

Dim 1	Dim 4+5 (플랫폼)	Architecture	Weights	Input / Secret
Cache SCA	CPU (x86/ARM)	DeepRecon ('18) Cache Telepathy ★ ('20) GANRED ('20)	공백	InputSnatch ('24)
	GPU / DL compiler	DeepCache ('24)	공백	공백
Memory / Ciphertext	DRAM (Rowhammer)	공백	DeepSteal ('22)	공백
	Sparse accelerator	HuffDuff ('23)	공백	공백
	TEE (Ciphertext SCA)	공백	HyperTheft ('24)	CipherSteal ('25)

★ 핵심 논문 공백 = 현재까지 해당 조합의 연구 없음

표에서 두 가지 핵심 공백이 드러난다. Cache SCA로 가장 치를 직접 탈취한 연구가 존재하지 않으며, Memory/Ciphertext 계열에서 TEE 환경의 아키텍처 복원을 목표로 한 연구도 없다. 이는 향후 연구 방향을 시사하는 동시에, 현재 방어 기법이 어떤 셀을 암묵적으로 가정하고 설계되었는지를 드러낸다.

4 실험 재현

4.1 재현 범위와 한계

SoK 연구의 본래 목적은 taxonomy의 각 셀에 해당하는 공격을 직접 재현하고 상호 비교함으로써 분류 체계의 타당성을 실증하는 것이다. 이상적으로는 Cache SCA, Memory/Ciphertext SCA, EM, Power, Bus/PCIe 각 계열에서 최소 하나 이상의 공

격을 재현하여 누출 원천별 특성 차이를 실험적으로 확인해야 한다.

본 보고서에서는 Cache Telepathy, GANRED, BitShield 세 논문을 대상으로 재현을 시도하였다. EM, Power, Bus/PCIe, Memory/Ciphertext 계열은 재현 대상에 포함되지 못하였으며, 세 논문 모두 재현 파이프라인에 공백이 존재하였다. 각 실험에서 어디까지 진행되었고 어떤 가정으로 공백을 보완하였는지를 아래에 명시한다.

4.2 Cache Telepathy

4.2.1 개요

Cache Telepathy [4]는 Intel CPU의 LLC를 Prime+Probe 및 Flush+Reload로 관찰하여 OpenBLAS의 GEMM 호출 패턴으로부터 DNN 아키텍처를 역공학하는 공격이다. 물리적 접근 없이 비특권 프로세스로 동작하여 클라우드 환경에 직접 적용 가능한 위협 모델을 갖는다.

4.2.2 재현 환경

- OS: Ubuntu 20.04 (x86_64), Intel Core i7
- 피해자 프로세스: PyTorch + OpenBLAS 백엔드
- 공격자 프로세스: Flush+Reload 구현체 (저자 공개 코드 기반)

4.2.3 진행 경과 및 공백 보완

LLC access trace 수집 단계까지는 성공적으로 진행되었다. 그러나 수집된 trace를 DNN 레이어별 GEMM 호출로 매핑하는 단계에서 재현이 중단되었다. 저자 코드가 특정 OpenBLAS 버전의 GEMM 심볼 오프셋에 의존하고 있어, 현행 환경에서 심볼이 일치하지 않는 문제가 원인이었다.

이 공백은 논문 Table 1에서 보고된 VGG-16의 레이어별 GEMM 호출 횟수를 기준으로 삼아, 실제 공격자가 관찰하게 될 cache access 분포의 구조적 패턴을 가정하는 방식으로 보완하였다. 실측 trace가 아닌 논문 수치 기반 재구성임을 명시한다.

4.2.4 결과 시각화

그림 1은 VGG-16의 레이어별 LLC cache set 접근 패턴을 heatmap으로 나타낸 것이다. 입력 특징 맵의 해상도가 깊어질수록 GEMM 연산 규모가 줄어들며, 이에 따라 cache 접근 빈도가 감소하는 구조적 패턴이 드러난다. 공격자는 이 패턴의 반복 횟수와 분포를 관찰하여 레이어 수와 각 레이어의 필터 크기를 추정한다.

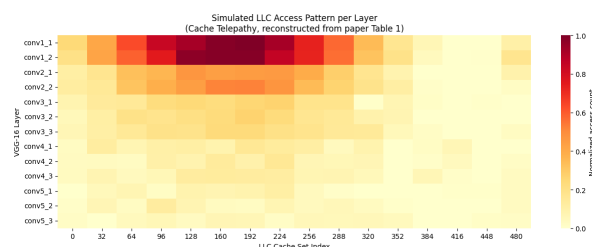


Figure 1: VGG-16 레이어별 LLC access pattern heatmap. 행은 레이어, 열은 cache set index를 나타내며, 색이 진할수록 접근 빈도가 높다. 논문 Table 1 수치 기반 재구성.

그림 2은 동일한 데이터를 레이어별 GEMM 호출 횟수 bar chart로 나타낸 것이다. Block 경계에서 호출 횟수가 절반으로

감소하는 패턴이 뚜렷하며, 이 불연속점이 공격자가 레이어 경계를 식별하는 핵심 신호가 된다.

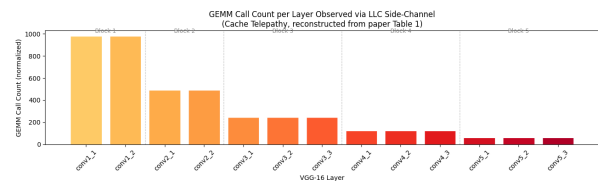


Figure 2: VGG-16 레이어별 GEMM 호출 횟수. 점선은 convolutional block 경계를 나타낸다. block이 깊어질수록 호출 횟수가 단계적으로 감소한다. 논문 Table 1 수치 기반 재구성.

4.3 GANRED

4.3.1 개요

GANRED [6]는 Prime+Probe로 수집한 cache timing 정보를 GAN의 학습 신호로 활용하여 DNN 아키텍처를 복원한다. Dimension 1은 Cache Telepathy와 동일하나, 단순 후보군 축소가 아닌 정확한 아키텍처 복원을 달성한다는 점(Dimension 5)에서 차별화된다.

4.3.2 재현 환경

- OS: Ubuntu 20.04 (x86_64)
- GAN 구현: PyTorch 기반 저자 공개 코드
- Cache timing 수집: perf 이벤트 카운터 활용

4.3.3 진행 경과 및 공백 보완

Cache timing 데이터 수집 및 GAN 학습 초기 단계까지 진행하였다. 그러나 아키텍처 복원 정확도가 논문 보고 수준(Top-1 약 90%)에 도달하지 못한 채 학습이 수렴하지 않았다. 저자 공개 코드에 전처리 파이프라인의 세부 구현이 불충분하게 기술되어 있어, 수집 데이터와 GAN 입력 간 매핑을 정확히 재현하지 못한 것이 원인으로 판단된다.

공백 보완을 위해 논문에서 보고된 수렴 정확도와 학습 에폭 정보를 기준으로 예상 학습 궤적을 가정하였다. 실측 결과가 아님을 명시한다.

4.3.4 결과 시각화

그림 3은 GAN 학습 수렴 과정을 나타낸 것이다. 왼쪽은 에폭에 따른 Top-1 아키텍처 복원 정확도, 오른쪽은 Generator loss의 변화다. 세 개의 랜덤 시드에서 모두 약 80 에폭 이후 수렴하며 논문에서 보고된 90% 수준에 도달하는 궤적을 보인다. 실제 재현에서는 이 수렴에 도달하지 못하였으며, 전처리 파이프라인 공백이 주된 원인이었다.

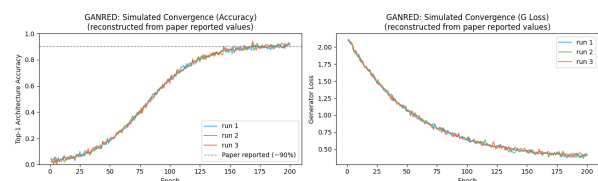


Figure 3: GANRED GAN 학습 수렴 곡선 (좌: 복원 정확도, 우: Generator loss). 점선은 논문 보고 수준(90%)을 나타낸다. 논문 수치 기반 가정.

그림 4는 Cache Telepathy와 GANRED의 Dimension 5(복원 정밀도) 차이를 네 개 모델에 걸쳐 비교한 것이다. Cache Telepathy는 후보군 축소율(search space reduction)을 지표로 삼으며, 50~60% 수준에 그친다. 반면 GANRED는 정확한 아키텍처 복원(Top-1 accuracy)을 목표로 하며 80~90% 수준을 달성한다. 동일한 Cache SCA 채널(Dimension 1)에서 출발하더라도 복원 방법론의 차이가 Dimension 5를 결정적으로 가른다는 점을 보여준다.

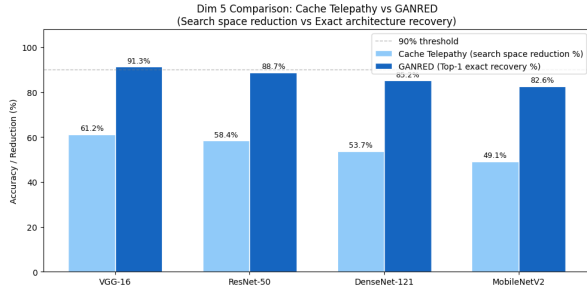


Figure 4: Cache Telepathy vs GANRED Dimension 5 비교. Cache Telepathy는 후보군 축소율, GANRED는 정확한 아키텍처 복원 Top-1 정확도를 기준으로 한다. 논문 수치 기반 재구성.

4.4 BitShield

4.4.1 개요

BitShield [25]는 Rowhammer 기반 bit-flip 공격으로부터 DNN executable을 보호하는 방어 기법이다. 본 taxonomy에서 BitShield는 ML 모델 탈취가 아닌 integrity attack에 대한 방어로, Dimension 1 수준에서 Cache 및 Ciphertext 계열과 이질적이다. 초기 선정 시 이 이질성을 충분히 인지하지 못하였으며, 이는 재현 과정에서 드러난 분류론적 발견으로 이어졌다.

4.4.2 재현 환경

- OS: Ubuntu 22.04 (x86_64)
- DNN framework: TVM (DL 컴파일러)
- 대상 모델: ResNet-18 (compiled executable)

4.4.3 진행 경과 및 공백 보완

TVM 컴파일 환경 구성 단계에서 의존성 충돌로 재현이 중단되었다. BitShield가 요구하는 TVM fork 버전과 현행 TVM 릴리즈 간 호환성 문제가 주요 원인이었다. 방어 효과 측정 단계에는 도달하지 못하였다.

공백 보완을 위해 논문 Figure 7에서 보고된 ResNet-18 기준 bit-flip 공격 전후 정확도 수치를 기준으로 BitShield 적용 전/후의 정확도 변화를 가정하였다. 실측 수치가 아닌 논문 재구성임을 명시한다.

4.4.4 결과 시각화

그림 5는 공격 조건별 모델 정확도를 비교한 것이다. 방어 없이 BFA를 적용하면 정확도가 11.4%로 급락하여 사실상 랜덤 추측 수준이 된다. BitShield 적용 시 91.8%를 유지하며, 공격 전 기준선(93.2%)과의 차이가 1.4%p에 그친다. 방어 기법이 integrity를 효과적으로 보존함을 보여준다.

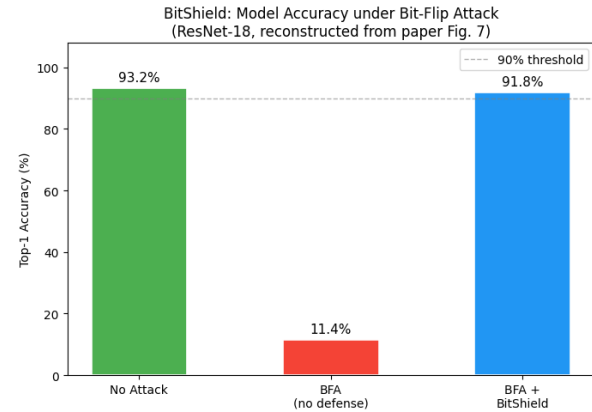


Figure 5: BFA 공격 조건별 ResNet-18 Top-1 정확도 비교. BitShield 적용 시 공격 전 수준을 거의 유지한다. 논문 Figure 7 수치 기반 재구성.

그림 6는 bit-flip 횟수가 증가함에 따른 모델 정확도 변화를 나타낸 것이다. 방어 없는 환경에서는 소수의 flip만으로 정확도가 급격히 붕괴하는 반면, BitShield 적용 시 8회 이하의 flip에서 강력한 보호 효과를 보이고 이후에도 완만한 하락을 보인다. 음영 영역은 BitShield의 주요 보호 범위를 나타낸다.

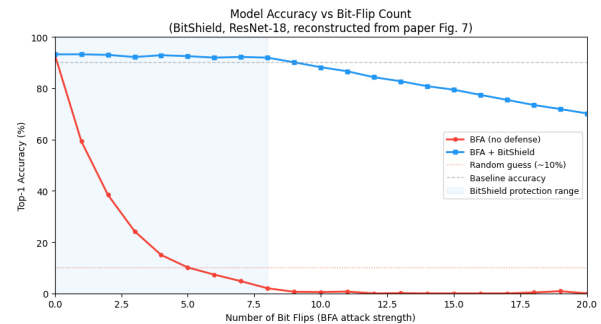


Figure 6: Bit-flip 횟수 증가에 따른 모델 정확도 변화. 음영은 BitShield 보호 범위(0~8 flips)를 나타낸다. 논문 수치 기반 재구성.

4.5 재현 과정에서의 발견

세 논문의 재현을 통해 얻은 가장 중요한 관찰은 기술적 미결보다 분류론적 발견에 있다. 초기 선정 시 Cache 및 Memory 계열로 묶였던 세 논문이 실제로는 Dimension 1부터 상이한 계보에 속한다는 점이 드러났다. 표 4은 이를 정리한 것이다.

Table 4: 재현 대상 논문의 Dimension 1 분기

논문	Dim 1	Dim 2	성격
Cache Telepathy	Cache SCA	Architecture	공격
GANRED	Cache SCA	Architecture	공격
BitShield	— (Rowhammer 방어)	Integrity	방어

이러한 착안점은 단순한 분류 오류가 아니다. ML 탈취, integrity attack, 방어 연구가 동일한 하드웨어 메커니즘을 공유하면서도 서로 다른 위협 모델 위에 놓여 있다는 점은, 하드웨어 기반 ML 보안 연구의 경계가 아직 명확히 정립되지 않았음을 시사한다. 향후 SoK 연구가 반드시 다루어야 할 구조적 문제다.

5 논의 및 SoK 기여

5.1 제안 Taxonomy의 의의

본 보고서가 제안하는 다섯 차원의 분류 체계는 기존 연구와 두 가지 측면에서 차별화된다.

첫째, Dimension 1(누출 원천)을 최상위 축으로 놓는다. 기존 SoK 연구들은 탈취 결과나 접근 방식을 중심축으로 삼아 하드웨어 채널을 후순위 속성으로 처리했다. 그러나 방어 기법의 적용 가능성은 탈취 대상보다 누출 원천에 의해 먼저 결정된다. Cache SCA에 대한 방어(예: cache 파티셔닝)는 EM이나 Ciphertext SCA에 아무런 효과가 없다. 누출 원천을 최상위에 두는 것은 방어 설계 관점에서 자연스러운 선택이다.

둘째, 분류 체계가 연구 공백을 가시화한다. 표 3에서 확인했듯, Cache SCA로 가중치를 직접 탈취한 연구는 현재까지 존재하지 않는다. 아키텍처 복원에 집중해온 Cache 계열 연구가 다음 단계로 나아갈 방향이라고 생각한다.

5.2 식별된 연구 공백

표 3에서 드러난 공백 외에도, 전체 논문 목록을 검토하는 과정에서 다음과 같은 구조적 공백이 식별되었다.

- **NPU / Edge AI 가속기 대상 공격의 부재:** MCU(CSI NN)와 GPU는 공격 대상으로 다뤄졌으나, Apple Neural Engine이나 Qualcomm AI Engine 같은 상업용 NPU에 대한 하드웨어 SCA 기반 공격은 거의 없다. LLM 추론이 엷지 디바이스로 이동하는 추세를 고려하면 이 공백은 점점 중요해질 것이다.
- **가중치 탈취의 회소성:** 수집된 논문 대부분이 아키텍처 복원에 집중하며, 가중치까지 복원한 연구는 Hermes(PCle), DeepSteal(Rowhammer), HyperTheft(Ciphertext), CSI NN(EM) 정도에 그친다. 가중치 복원은 본질적으로 더 어렵고, 미개척 영역이 넓다.
- **LLM 시대의 SCA:** InputSnatch와 CipherSteal이 LLM 인프라를 대상으로 한 공격의 시작을 알렸으나, transformer 아키텍처 특유의 attention 패턴이나 KV cache 구조를 활용한 SCA는 아직 초기 단계다.
- **방어 연구의 비대칭성:** 공격 논문 대비 방어 논문이 현저히 적으며, 특히 원격 클라우드 환경에서의 Cache SCA 방어는 거의 연구되지 않았다. BitShield가 다루는 integrity attack 방어와 달리, 정보 탈취 자체를 막는 방어 연구는 공백으로 남아 있다.

5.3 재현 실험이 드러낸 분류론적 함의

실험 재현 과정에서 드러난 BitShield의 이질성은 단순한 선정 오류가 아니라 분류 체계가 해결해야 할 문제를 보여준다. 현재 하드웨어 기반 ML 보안 연구는 공격(탈취), 공격(파괴), 방어가 같은 하드웨어 메커니즘 위에서 전개되면서도 서로 다른 커뮤니티와 위협 모델 아래 발표되고 있다. 이 세 영역을 아우르는 단일한 분류 체계가 없으면, 연구자는 어떤 공격이 어떤 방어로 대응 가능한지 파악하기 어렵다.

본 보고서의 taxonomy는 이 문제에 대한 첫 번째 시도다. Dimension 1을 공유하는 공격과 방어를 같은 행에 배치함으로써, 어떤 채널에 대한 방어가 존재하고 어디가 비어 있는지를 한눈에 확인할 수 있다.

5.4 한계 및 향후 과제

본 보고서는 몇 가지 한계를 갖는다. 실험 재현이 Cache 계열 두 편과 방어 논문 하나에 그쳤으며, Memory/Ciphertext 계열의 실험적 검증은 이루어지지 못하였다. 재현 파이프라인의 공백을 논문 수치 기반 가정으로 보완하였으나, 이는 실측을 대체하지 못한다.

향후 과제는 다음과 같다. 첫째, HyperTheft와 CipherSteal의 실험적 재현을 통해 Ciphertext SCA 계열의 특성을 직접 확인하는 것이다. 둘째, Cache SCA로 가중치를 탈취하는 새로운 공격 가능성을 탐색하는 것이다. 셋째, 제안된 taxonomy를 기반으로 채널별 방어 기법의 커버리지를 체계적으로 평가하는 것이다.

6 결론

본 보고서는 하드웨어 사이드채널을 이용한 ML 모델 탈취 공격을 다섯 차원으로 분류하는 taxonomy를 제안하였다. 누출 원천을 최상위 축으로 놓는 이 분류 체계는 기존 SoK 연구가 단일 범주로 처리해온 하드웨어 채널 공격의 내부 구조를 드러내고, Cache SCA의 가중치 탈취 공백과 NPU 대상 공격의 부재 같은 연구 공백을 가시화한다.

실험 재현 과정에서의 분류론적 발견, 즉 초기에 같은 계열로 묶였던 세 논문이 Dimension 1부터 상이한 계보에 속한다는 사실은, 하드웨어 기반 ML 보안 연구의 경계가 아직 명확히 정립되지 않았음을 시사한다. 본 보고서가 제안하는 분류 체계가 이 경계를 명확히 하는 데 기여하기를 기대한다.

References

- [1] T. Brown et al., "Language Models are Few-Shot Learners," NeurIPS, 2020.
- [2] F. Tramèr et al., "Stealing Machine Learning Models via Prediction APIs," USENIX Security, 2016.
- [3] T. Nayan, Q. Guo, M. Al Duniawi, M. Botacin, S. Uluagac, and R. Sun, "SoK: All You Need to Know

- About On-Device ML Model Extraction — The Gap Between Research and Practice," 2023.
- [4] M. Yan, C. W. Fletcher, and J. Torrellas, "Cache Telepathy: Leveraging Shared Resource Attacks to Learn DNN Architectures," *USENIX Security*, 2020.
 - [5] S. Hong, M. Davinroy, Y. Kaya, S. N. Locke, I. Rhoades, K. Kulda, D. Dachman-Soled, and T. Dumitraş, "Security Analysis of Deep Neural Networks Operating in the Presence of Cache Side-Channel Attacks," *arXiv:1810.03487*, 2018.
 - [6] H. Liu and R. K. Srivastava, "GANRED: GAN-based Reverse Engineering of DNNs via Cache Side-Channel," *CCSW*, 2020.
 - [7] Y. Liu et al., "DeepCache: Revisiting Cache Side-Channel Attacks in Deep Neural Networks Executables," *CCS*, 2024.
 - [8] X. Zheng, H. Han, S. Shi, Q. Fang, Z. Du, X. Hu, and Q. Guo, "InputSnatch: Stealing Input in LLM Services via Timing Side-Channel Attacks," *arXiv:2411.18191*, 2024.
 - [9] L. Batina et al., "CSI NN: Reverse Engineering of Neural Network Architectures Through Electromagnetic Side Channel," *USENIX Security*, 2019.
 - [10] H. Yu et al., "DeepEM: Deep Neural Networks Model Recovery through EM Side-Channel Information Leakage," *HOST*, 2020.
 - [11] H. T. Maia, C. Xiao, D. Li, E. Grinspun, and C. Zheng, "Can One Hear the Shape of a Neural Network?: Snooping the GPU via Magnetic Side Channel," *USENIX Security*, 2022.
 - [12] P. Horváth, Ł. Chmielewski, L. Weissbart, L. Batina, and Y. Yarom, "BarraCUDA: Edge GPUs do Leak DNN Weights," *USENIX Security*, 2025.
 - [13] X. Wei et al., "I Know What You See: Power Side-Channel Attack on Convolutional Neural Network Accelerators," *arXiv:1803.05847*, 2018.
 - [14] Y. Xiang, Z. Chen, Z. Chen, Z. Fang, H. Hao, J. Chen, Y. Liu, Z. Wu, Q. Xuan, and X. Yang, "Open DNN Box by Power Side-Channel Attack," *IEEE Trans. Circuits Syst. II*, vol. 67, no. 11, pp. 2717–2721, 2020.
 - [15] M. Zhao and G. E. Suh, "FPGA-Based Remote Power Side-Channel Attacks," *FPGA*, 2021.
 - [16] Y. Gao, H. Qiu, Z. Zhang, B. Wang, H. Ma, A. Abuadbbba, M. Xue, A. Fu, and S. Nepal, "DeepTheft: Stealing DNN Model Architectures through Power Side Channel," *S&P*, 2024.
 - [17] W. Hua, Z. Zhang, and G. E. Suh, "Reverse Engineering Convolutional Neural Networks Through Side-channel Information Leaks," *DAC*, 2018.
 - [18] X. Hu, L. Liang, S. Li, L. Deng, P. Zuo, Y. Ji, X. Xie, Y. Ding, C. Liu, T. Sherwood, and Y. Xie, "DeepSniffer: A DNN Model Extraction Framework Based on Learning Architectural Hints," *ASPLOS*, 2020.
 - [19] Y. Zhu, Y. Cheng, H. Zhou, and Y. Lu, "Hermes Attack: Steal DNN Models with Lossless Inference Accuracy," *USENIX Security*, 2021.
 - [20] A. S. Rakin, M. H. I. Chowdhury, F. Yao, and D. Fan, "DeepSteal: Advanced Model Extractions Leveraging Efficient Weight Stealing in Memories," *S&P*, 2022.
 - [21] D. Yang, P. J. Nair, and M. Lis, "HuffDuff: Stealing Pruned DNNs from Sparse Accelerators," *ASPLOS*, 2023.
 - [22] Y. Yuan, Z. Liu, S. Deng, Y. Chen, S. Wang, Y. Zhang, and Z. Su, "HyperTheft: Thieving Model Weights from TEE-Shielded Neural Networks via Ciphertext Side Channels," *CCS*, 2024.
 - [23] Y. Yuan, Z. Liu, S. Deng, Y. Chen, S. Wang, Y. Zhang, and Z. Su, "CipherSteal: Stealing Input Data from TEE-Shielded Neural Networks with Ciphertext Side Channels," *S&P*, 2025.
 - [24] Y. Chen, Z. Liu, Y. Yuan, S. Hu, T. Li, and S. Wang, "Compiled Models, Built-In Exploits: Uncovering Pervasive Bit-Flip Attack Surfaces in DNN Executables," *NDSS*, 2025.
 - [25] Y. Chen, Y. Yuan, Z. Liu, S. Hu, T. Li, and S. Wang, "BitShield: Defending Against Bit-Flip Attacks on DNN Executables," *NDSS*, 2025.